

RAPPORT DE STAGE

Deuxième année — BTS SIO option SLAM

Cadilly LTD

Effectué par :

ANDRIANAMBININTSOA Benja Manjaka Ginola

Période :

Du 20 janvier au 2 mars 2026

Encadrement :

Tuteur en entreprise : Madame Huda Chowkowsky

Tuteur académique : Monsieur Yussuf

MCCI BUSINESS SCHOOL - Cyber Cité d'Ebène, Île Maurice

CADILLY LTD – Tribeca Trianon, Île Maurice

Année universitaire 2025 — 2026

REMERCIEMENTS

Je tiens tout d'abord à exprimer ma sincère gratitude à Madame Huda Chowkowsky et Monsieur Junaïd, mes tuteurs en entreprise, pour leur disponibilité, leur patience et leurs conseils avisés tout au long de ce stage. Leur accompagnement m'a permis de mieux comprendre les enjeux du terrain et de mener à bien les missions qui m'ont été confiées.

Je remercie également Monsieur Yussuf, mon tuteur académique, pour son suivi régulier et ses remarques constructives qui ont contribué à faire progresser mon projet.

Mes remerciements vont aussi à toute l'équipe de Cadilly LTD, et particulièrement au personnel du restaurant Feu de Bois, pour leur accueil chaleureux et la confiance qu'ils m'ont accordée durant ces semaines de stage.

Enfin, je tiens à remercier mon binôme de projet avec qui j'ai eu le plaisir de collaborer. La répartition des tâches et les échanges que nous avons eus ont été une vraie source d'apprentissage.

RESUME

Ce rapport présente le travail effectué lors de mon stage de deuxième année au sein de l'entreprise Cadilly LTD, réalisé du 20 janvier au 2 mars 2026. La mission consistait à concevoir et développer une application de gestion de stock pour le restaurant Feu de Bois, qui fonctionnait jusque-là avec un suivi papier complété par quelques fichiers Excel.

Le projet a été mené en binôme. J'ai pris en charge la partie authentification et gestion de session, utilisateurs et ses privilège la modélisation de la base de données, les fonctionnalités CRUD ainsi qu'une partie des interfaces JavaFX et de la logique métier liée aux mouvements de stock. Le travail s'est articulé autour de plusieurs étapes : recueil des besoins, modélisation avec MERISE et UML, choix des technologies, développement, tests et livraison.

L'application a été livrée dans les délais. Elle fonctionne en local sur les postes du restaurant et se synchronise avec une base de données PostgreSQL hébergée en ligne, ce qui permet un accès cohérent aux données depuis plusieurs postes. Les objectifs fixés au départ ont été atteints et l'outil est aujourd'hui opérationnel.

SOMMAIRE

INTRODUCTION	1
1. Présentation de l'entreprise : Cadilly LTD.....	1
2. Contexte du stage	1
3. Objectifs du stage.....	2
I. Présentation du projet	3
A. Contexte	3
B. Objectifs du projet.....	3
C. Cahier des charges.....	4
II. Déroulement du stage.....	5
A. Première partie (Partie 1).....	5
1. Collecte des besoins	5
2. Analyse et conception (MERISE / UML)	5
3. Présentation du workflow.....	5
B. Deuxième partie (Partie 2)	7
1. Modélisation de la base de données (MCD)	7
2. Diagramme de classes (UML)	8
3. Choix technologiques	9
4. Présentation des entités et règles de gestion	10
C. Troisième partie (Partie 3)	12
1. Développement du projet.....	12
2. Tests et débogage.....	13
3. Déploiement et livraison	14
III. Résultats obtenus	15
A. Présentation de l'application	15
B. Évaluation des résultats	16
CONCLUSION et PERSPECTIVES.....	17

Table des figures

Figure 1 : Modèle Conceptuel de Données (MCD)	7
Figure 2 : Diagramme de classes UML	8
Figure 3 : Page de connexion	12
Figure 4 : Tableau de bord (Dashboard).....	12
Figure 5 : Liste des produits	13
Figure 6 : Écran d'entrée de stock	13
Figure 7 : Écran de sortie de stock.....	14
Figure 8 : Page Inventaire.....	15
Figure 9 : Page Historique	15

INTRODUCTION

1. Présentation de l'entreprise : Cadilly LTD

Cadilly LTD est une entreprise mauricienne active dans le secteur de la restauration et des services associés. Elle exploite notamment le restaurant Feu de Bois, un établissement reconnu pour sa cuisine authentique et son ambiance conviviale. Le restaurant propose une carte variée combinant plats traditionnels et créations plus modernes, avec une clientèle à la fois locale et internationale.

Comme beaucoup d'établissements du même type, Cadilly LTD doit gérer au quotidien un volume important d'ingrédients, de boissons, de produits d'entretien et de consommables. Cette gestion est essentielle pour garantir la qualité du service, éviter les ruptures et maîtriser les coûts. C'est dans ce contexte que l'entreprise m'a accueilli pour mon stage de deuxième année.

2. Contexte du stage

Avant mon arrivée, la gestion du stock du restaurant reposait principalement sur un suivi papier, complété par quelques fichiers Excel tenus à jour manuellement. Cette organisation, bien que fonctionnelle au quotidien, présentait plusieurs limites. Les erreurs de saisie étaient fréquentes, le suivi des entrées et des sorties n'était pas toujours fiable, et il était difficile de savoir à un instant donné quel était l'état réel du stock.

La direction souhaitait donc moderniser cette gestion en mettant en place un outil informatique simple, accessible au personnel et capable de tracer toutes les opérations. L'idée n'était pas de transformer complètement l'organisation du travail, mais plutôt d'apporter un support fiable qui réduirait les erreurs et donnerait une meilleure visibilité sur les stocks.

C'est ce besoin concret qui a donné naissance au projet. Le stage s'est déroulé sur une période de six semaines, du 20 janvier au 2 mars 2026, au sein de Cadilly LTD, sous la responsabilité de Madame Huda CHOWKOWRY et Monsieur Junaid en entreprise et de Monsieur Yussuf côté académique.

3. Objectifs du stage

L'objectif principal du stage était de concevoir et développer une application de gestion de stock adaptée aux besoins réels du restaurant Feu de Bois. L'application devait être simple à utiliser, puisque les personnes amenées à s'en servir ne sont pas forcément à l'aise avec l'outil informatique. Elle devait également être fiable, car une mauvaise information sur les stocks peut avoir des conséquences directes sur le service.

Au-delà de l'aspect technique, le stage avait aussi pour but de me confronter à un projet en conditions réelles : comprendre un besoin exprimé par un client, le traduire en spécifications, concevoir une architecture, coder, tester, puis livrer un produit fini. Le travail en binôme ajoutait une dimension de coordination et de partage du code qui m'a beaucoup appris.

Enfin, un objectif plus personnel était de consolider mes compétences en Java, en base de données et en conception logicielle, tout en découvrant de nouveaux outils comme JavaFX et le pattern DAO.

I. Présentation du projet

A. Contexte

Le projet, nommé en interne « Gestion Stock FDB » (pour Feu de Bois), est né d'un constat partagé par l'équipe : la gestion manuelle des stocks devenait de plus en plus lourde à mesure que l'activité du restaurant progressait. Les commandes fournisseurs, les sorties liées au service, les inventaires de fin de mois... tout cela était géré avec des tableaux papier et quelques fichiers Excel, ce qui entraînait régulièrement des oublis ou des incohérences.

L'idée était donc de proposer un outil qui centralise l'information, qui garde une trace de chaque opération et qui permette à n'importe quel membre autorisé de l'équipe de consulter facilement l'état du stock. Le projet devait rester à une échelle raisonnable, avec des fonctionnalités utiles au quotidien plutôt qu'un système complexe difficile à prendre en main.

B. Objectifs du projet

Le projet poursuivait plusieurs objectifs concrets. D'abord, permettre à l'équipe d'enregistrer facilement les entrées et sorties de stock, avec un motif et une traçabilité de l'utilisateur. Ensuite, offrir une vue synthétique de l'état du stock, avec des alertes automatiques lorsque la quantité d'un produit passe sous un seuil défini. Enfin, pouvoir consulter l'historique des mouvements pour comprendre l'évolution d'un produit dans le temps.

L'application devait aussi intégrer un système de connexion avec gestion des rôles, de manière à ce que certaines actions sensibles (suppression de produit, gestion des utilisateurs) soient réservées aux personnes autorisées. Côté technique, l'outil devait fonctionner sur les postes du restaurant sans nécessiter d'installation complexe, tout en partageant les données avec une base centralisée pour que plusieurs utilisateurs puissent travailler de manière cohérente.

C. Cahier des charges

Le cahier des charges a été construit à partir des entretiens menés avec la tutrice et l'équipe du restaurant. Il peut se résumer autour des points suivants :

- Authentification sécurisée avec gestion de rôles .
- Gestion complète des produits : ajout, modification, suppression, recherche et filtrage par catégorie.
- Enregistrement des entrées et sorties de stock avec motif, date et utilisateur associés.

- Affichage d'un tableau de bord donnant une vue d'ensemble : nombre de produits, alertes, statuts.
- Page d'inventaire détaillée avec un statut visuel.
- Historique consultable des mouvements, avec possibilité de filtrer par période ou par produit.
- Export des données au format CSV ou Excel pour archivage ou analyse externe.
- Base de données hébergée en ligne pour permettre la synchronisation entre plusieurs postes.

Côté technologique, le choix s'est porté sur Java avec JavaFX pour l'interface graphique, et PostgreSQL pour la base de données.

II. Déroulement du stage

A. Première partie

1. Collecte des besoins

La première étape du stage a été consacrée à comprendre précisément ce que l'équipe attendait de l'application. Pour cela, j'ai mené plusieurs entretiens informels avec la tutrice et le personnel du restaurant. Ces échanges ont permis de dégager les vrais besoins du terrain, qui ne sont pas toujours ceux qu'on imagine au départ.

Par exemple, il est rapidement apparu que la notion de « seuil minimum » était essentielle : plutôt que de lister simplement les quantités, l'équipe voulait être alertée quand un produit approchait de la rupture. De la même façon, la traçabilité des sorties (qui a fait quoi et quand) était importante pour comprendre d'éventuelles incohérences lors de l'inventaire.

2. Analyse et conception (MERISE / UML)

Une fois les besoins cernés, je suis passé à la phase de modélisation. J'ai utilisé la méthode MERISE pour concevoir la structure de la base de données à travers un Modèle Conceptuel de Données (MCD), et la notation UML pour représenter l'architecture logicielle via un diagramme de classes. Ces deux modèles ont servi de référence tout au long du développement et seront présentés en détail dans la partie B.

3. Présentation du workflow

Pour bien mesurer l'apport de l'application voici un aperçu du Workflow

Workflow cible (avec l'application) :

1. Connexion de l'utilisateur avec son identifiant et son mot de passe.
2. Réception d'une livraison : saisie directe dans l'écran « Entrée de stock » avec le produit et la quantité.

3. Mise à jour automatique du stock dans la base, visible immédiatement sur tous les postes.
4. Sortie pour le service : enregistrement rapide dans l'écran « Sortie de stock » avec un motif.
5. Alerte automatique quand un produit passe sous son seuil minimum.
6. Inventaire et historique consultables à tout moment, avec possibilité d'exporter les données.

B. Deuxième partie

1. Modélisation de la base de données (MCD)

Le Modèle Conceptuel de Données a été la première étape concrète de la conception. Il permet de représenter, de façon indépendante de toute technologie, les différentes entités manipulées par le système et les relations qui existent entre elles.

2. Diagramme de classes (UML)

En parallèle du MCD, j'ai réalisé un diagramme de classes UML pour représenter l'architecture logicielle de l'application. Ce diagramme met en évidence les principales classes métier (Produit, Utilisateur, MouvementStock, Catégorie), les classes d'accès aux données (ProduitDAO, MouvementStockDAO), les services (SessionManager, AuthenticationService, StockManager) ainsi que la classe technique DatabaseConnection.

L'architecture repose sur le pattern MVC (Modèle-Vue-Contrôleur), complété par le pattern DAO (Data Access Object) pour isoler la couche de persistance. Deux classes utilisent le pattern Singleton : SessionManager, qui maintient la session de l'utilisateur connecté, et DatabaseConnection, qui centralise l'accès à la base.

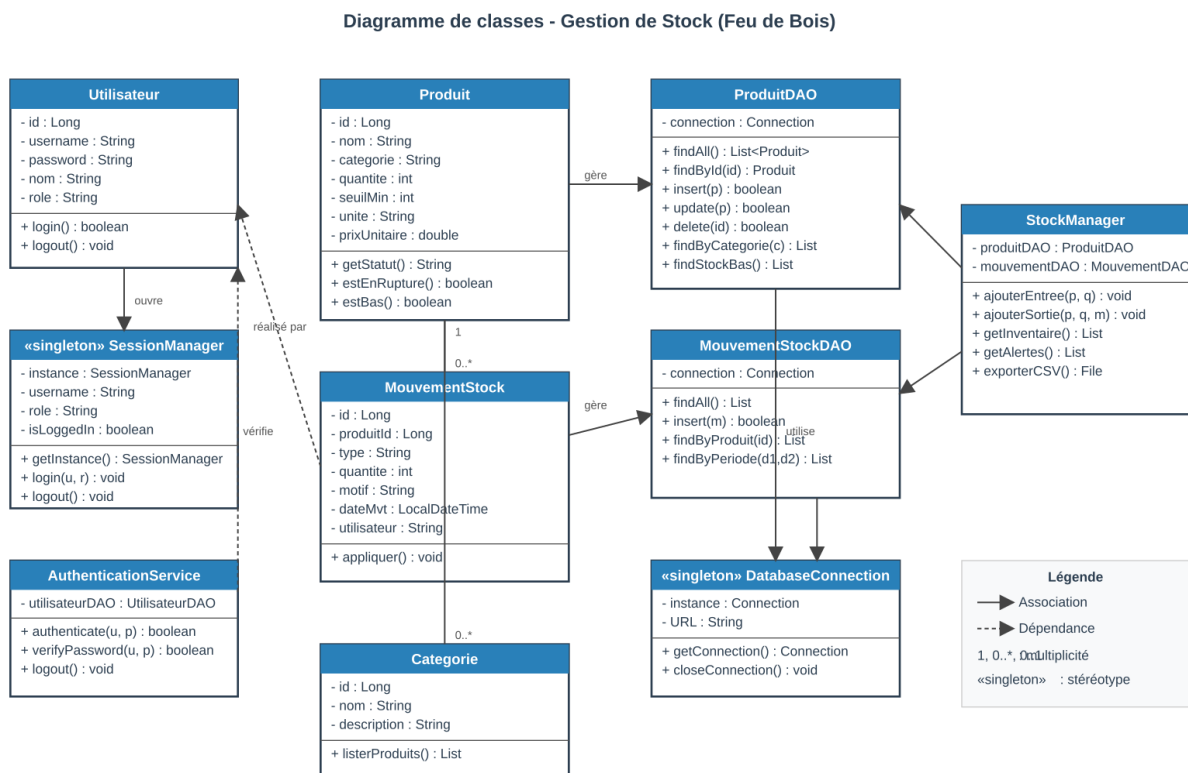


Figure 2 : Diagramme de classes UML

Cette organisation présente plusieurs avantages. Elle rend le code plus lisible en séparant clairement les responsabilités, elle facilite les évolutions futures puisqu'une modification dans une couche n'impacte pas forcément les autres, et elle rend les tests plus simples car chaque composant peut être isolé.

3. Choix technologiques

Le choix des technologies a été guidé par plusieurs critères : la maîtrise que j'avais déjà des outils, la simplicité de déploiement sur les postes du restaurant, la qualité de l'interface graphique, et la capacité à fonctionner aussi bien en local qu'avec une base distante.

Les technologies retenues sont les suivantes :

- Java 17 comme langage principal, pour sa fiabilité, sa portabilité et sa richesse en bibliothèques.
- JavaFX pour l'interface graphique, avec les fichiers FXML décrivant les vues et les contrôleurs gérant la logique.
- PostgreSQL comme système de gestion de base de données, hébergé sur le service Alwaysdata pour permettre la synchronisation entre postes.
- JDBC pour la connexion entre Java et PostgreSQL, sans passer par un ORM lourd afin de garder le contrôle sur les requêtes.
- Maven pour la gestion des dépendances et la construction du projet.
- IntelliJ IDEA comme environnement de développement.
- Git et GitHub pour le versionnement du code et le travail en binôme.

Ce choix permet d'avoir une application desktop réactive, avec une interface soignée grâce à JavaFX, tout en conservant les données dans une base relationnelle accessible depuis n'importe quel poste autorisé.

4. Présentation des entités et règles de gestion

Les utilisateurs :

L'application distingue plusieurs rôles d'utilisateur. Le propriétaire dispose de tous les droits, y compris la gestion des autres comptes. Le responsable peut gérer les produits et consulter les historiques, mais ne peut pas créer de nouveaux comptes. L'employé, quant à lui, est limité aux opérations quotidiennes : consultation du stock, enregistrement des entrées et sorties. Cette séparation des droits est gérée par le SessionManager, qui garde en mémoire le rôle de l'utilisateur connecté, et est vérifiée avant chaque action sensible.

Les produits :

Chaque produit est caractérisé par un nom, une catégorie, une quantité en stock, un seuil minimum, une unité (kg, litre, pièce...) et un prix unitaire. Les catégories utilisées correspondent aux besoins du restaurant : Beverages (boissons), Meat (viandes), Dairy Product (produits laitiers), Dry Items (produits secs), Cleaning Products (produits d'entretien)

et Other Items (divers). Cette classification facilite la navigation dans la liste des produits et permet de générer des statistiques par famille.

Un produit peut avoir trois statuts automatiques calculés à partir de sa quantité et de son seuil : OK (stock suffisant), Stock bas (quantité $\leq$ seuil minimum) ou Rupture (quantité = 0). Ces statuts sont affichés avec des couleurs distinctes dans l'application pour une lecture rapide.

Les mouvements :

Chaque opération sur le stock génère un mouvement enregistré en base. Un mouvement comporte un type (ENTRÉE ou SORTIE), le produit concerné, la quantité, un motif (livraison fournisseur, service, casse, inventaire...), la date et l'utilisateur qui l'a effectué. Cette table est centrale pour la traçabilité : c'est elle qui permet ensuite de reconstituer l'historique et d'expliquer d'éventuels écarts à l'inventaire.

Règles de gestion principales :

- Toute sortie est refusée si la quantité demandée dépasse la quantité disponible.
- Le stock est mis à jour automatiquement après chaque mouvement validé.
- Un produit ne peut pas être supprimé s'il a des mouvements associés ; il peut seulement être désactivé.
- Les opérations de gestion d'utilisateurs sont réservées au rôle « propriétaire ».
- Les mots de passe sont hachés avant enregistrement en base.

C. Troisième partie (Partie 3)

1. Développement du projet

La phase de développement s'est déroulée en parallèle pour les deux membres du binôme, chacun ayant sa zone de responsabilité. Personnellement, je me suis principalement occupé des éléments suivants.

Module d'authentification :

J'ai développé l'écran de connexion ainsi que toute la logique associée. L'utilisateur saisit son identifiant et son mot de passe, qui sont vérifiés par la classe `AuthenticationService`. En cas de succès, la classe `SessionManager` (implémentée en Singleton) conserve les informations de la session : nom d'utilisateur, rôle et état de connexion. Ces informations sont ensuite utilisées dans toute l'application pour afficher le bon menu et autoriser ou non certaines actions.

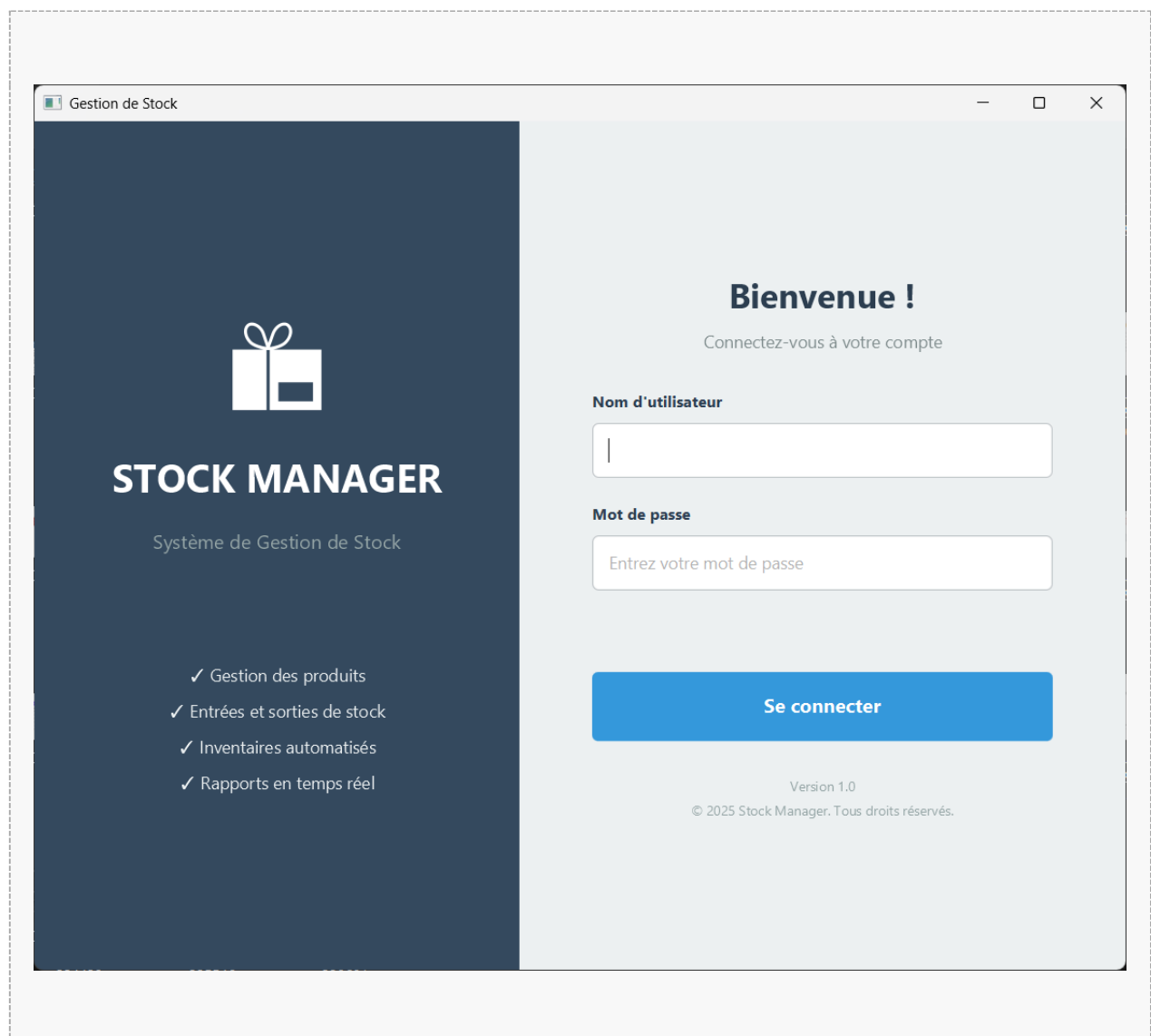


Figure 3 : Page de connexion

Tableau de bord :

Le tableau de bord est la page d'accueil après connexion. Il affiche en temps réel les principaux indicateurs : nombre total de produits, nombre de produits en alerte, nombre de catégories et dernières opérations effectuées. Les données sont récupérées directement depuis la base via les DAO, ce qui garantit que les chiffres affichés sont toujours à jour.

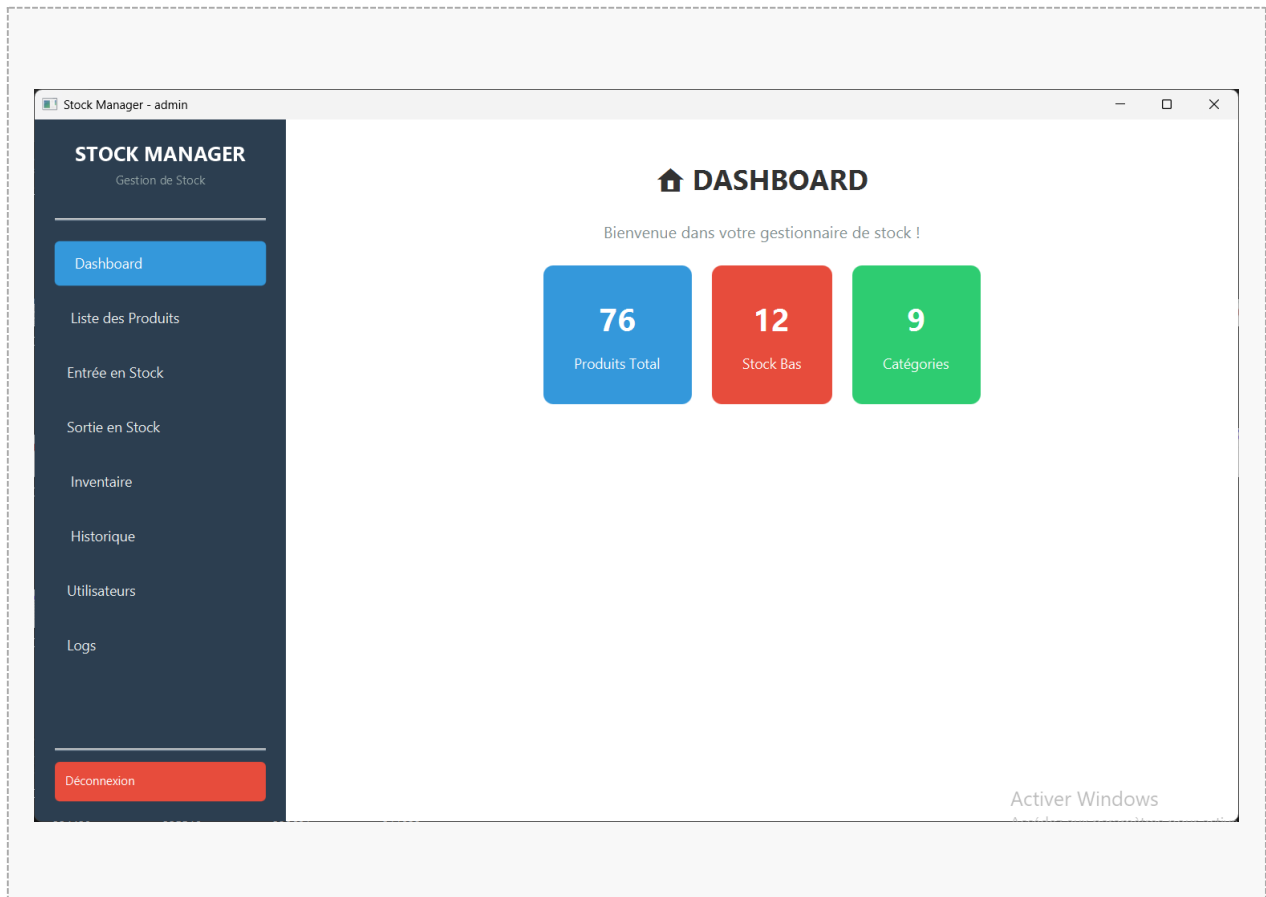


Figure 4 : Tableau de bord (Dashboard)

Gestion des produits :

L'écran de gestion des produits permet d'effectuer les opérations CRUD (Create, Read, Update, Delete) sur la table des produits. Une barre de recherche et un filtre par catégorie permettent de retrouver rapidement un élément dans la liste. Les modifications sont enregistrées directement en base grâce au ProduitDAO.

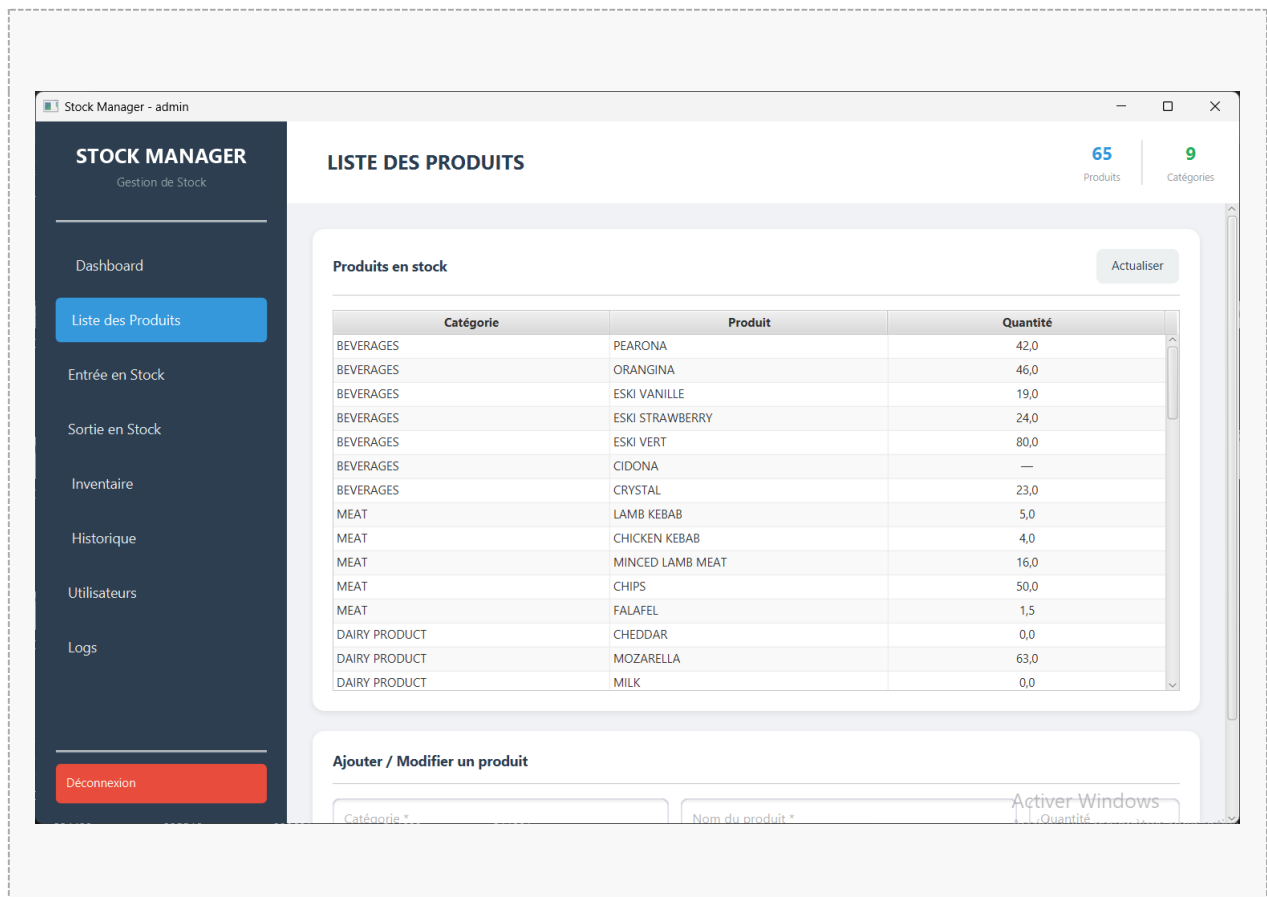


Figure 5 : Liste des produits

Entrées et sorties de stock :

Deux écrans distincts sont dédiés aux mouvements de stock. L'écran « Entrée de stock » sert à enregistrer les livraisons ou tout apport de marchandises. L'écran « Sortie de stock » gère les retraits liés au service ou à d'autres motifs (casse, inventaire...). Dans les deux cas, l'utilisateur sélectionne un produit dans une liste à recherche en temps réel, saisit la quantité et valide. Le système met immédiatement à jour la quantité en base et enregistre le mouvement dans l'historique.

The screenshot shows the 'ENTRÉE EN STOCK' (Stock Entry) interface. The left sidebar contains the 'STOCK MANAGER' logo and navigation links: Dashboard, Liste des Produits, Entrée en Stock (highlighted), Sortie en Stock, Inventaire, Historique, Utilisateurs, and Logs. A 'Déconnexion' button is at the bottom. The main area has a green header with a downward arrow and the text 'ENTRÉE EN STOCK' and 'Enregistrez les produits qui entrent dans le stock'. Below this is a 'Formulaire d'entrée' (Entry Form) with fields for 'Rechercher un produit' (Search for a product), 'Sélectionner le produit' (Select the product), 'Stock actuel' (Current stock), 'Nouveau stock' (New stock), 'Quantité à ajouter' (Quantity to add), and 'Motif (optionnel)' (Optional reason). A green 'Ajouter au Stock' button is at the bottom of the form. Below the form is a 'Historique des Entrées' (Entry History) section with a filter bar and a table showing the entry history.

Date	Produit	Quantité	Motif
25/03/2026 09:01	LAMB KEBAB	+ 3.0	

Figure 6 : Écran d'entrée de stock

The screenshot shows the 'SORTIE DE STOCK' (Stock Exit) interface. The left sidebar is identical to the previous screenshot, with 'Sortie en Stock' highlighted. The main area has a red header with an upward arrow and the text 'SORTIE DE STOCK' and 'Enregistrez les produits qui quittent le stock'. Below this is a 'Formulaire de sortie' (Exit Form) with fields for 'Rechercher un produit' (Search for a product), 'Sélectionner le produit' (Select the product), 'Stock actuel' (Current stock), 'Nouveau stock' (New stock), 'Quantité à retirer' (Quantity to remove), and 'Motif (optionnel)' (Optional reason). A red 'Enregistrer la Sortie' button is at the bottom of the form. Below the form is a 'Historique des mouvements' (Movement History) section with a filter bar and a table showing the movement history.

Date	Type	Produit	Quantité	Motif
18/03/2026 16:03		PEARONA	- 10.0	

Figure 7 : Écran de sortie de stock

2. Tests et débogage

Le développement a été ponctué de phases de tests réguliers. J'ai commencé par des tests manuels sur chaque écran, en vérifiant que les cas normaux fonctionnaient correctement puis en essayant de provoquer des erreurs (saisie vide, quantité négative, produit inexistant, connexion perdue...).

Plusieurs bugs ont été identifiés et corrigés au fil du développement. Parmi eux, une erreur de compilation Java liée à un caractère parasite dans la classe SessionManager, un problème de sensibilité à la casse sur les noms de fichiers, ou encore des soucis de synchronisation lorsque deux utilisateurs modifiaient le même produit en même temps. Chaque correction a été validée par un nouveau test avant passage à la fonctionnalité suivante.

À la fin du développement, des tests utilisateurs ont été organisés avec la tutrice et un employé du restaurant. Ces tests ont permis d'affiner certaines ergonomies (tailles de boutons, messages de confirmation) et de s'assurer que l'outil était utilisable par une personne peu habituée à l'informatique.

3. Déploiement et livraison

Le déploiement a été réalisé en deux temps. D'abord, la base de données PostgreSQL a été mise en ligne sur le service Alwaysdata, avec les tables créées à partir du MCD et quelques données de test. Les identifiants de connexion à la base sont stockés dans un fichier de configuration propre à chaque poste, ce qui évite de les retrouver en dur dans le code.

Ensuite, l'application a été packagée sous forme de fichier exécutable et installée sur les postes du restaurant. Le lancement se fait depuis un simple raccourci, et la connexion à la base est automatique au démarrage. Si la connexion échoue, un message clair invite l'utilisateur à vérifier sa connexion Internet.

Un petit manuel d'utilisation a été remis à l'équipe à la fin du stage. Il reprend les principales actions : se connecter, ajouter un produit, enregistrer une entrée, enregistrer une sortie, consulter l'inventaire et exporter les données. Ce document est volontairement court et imagé pour rester accessible.

III. Résultats obtenus

A. Présentation de l'application

À la fin du stage, l'application était entièrement fonctionnelle et déployée sur les postes du restaurant. Elle couvre l'ensemble des fonctionnalités prévues dans le cahier des charges et répond aux besoins exprimés lors de la phase de collecte.

La page d'inventaire offre une vue claire de l'état du stock, avec un code couleur pour identifier rapidement les produits en alerte. La page d'historique permet de remonter dans le temps et de filtrer les mouvements par date, produit ou utilisateur, ce qui est particulièrement utile lors des inventaires mensuels.

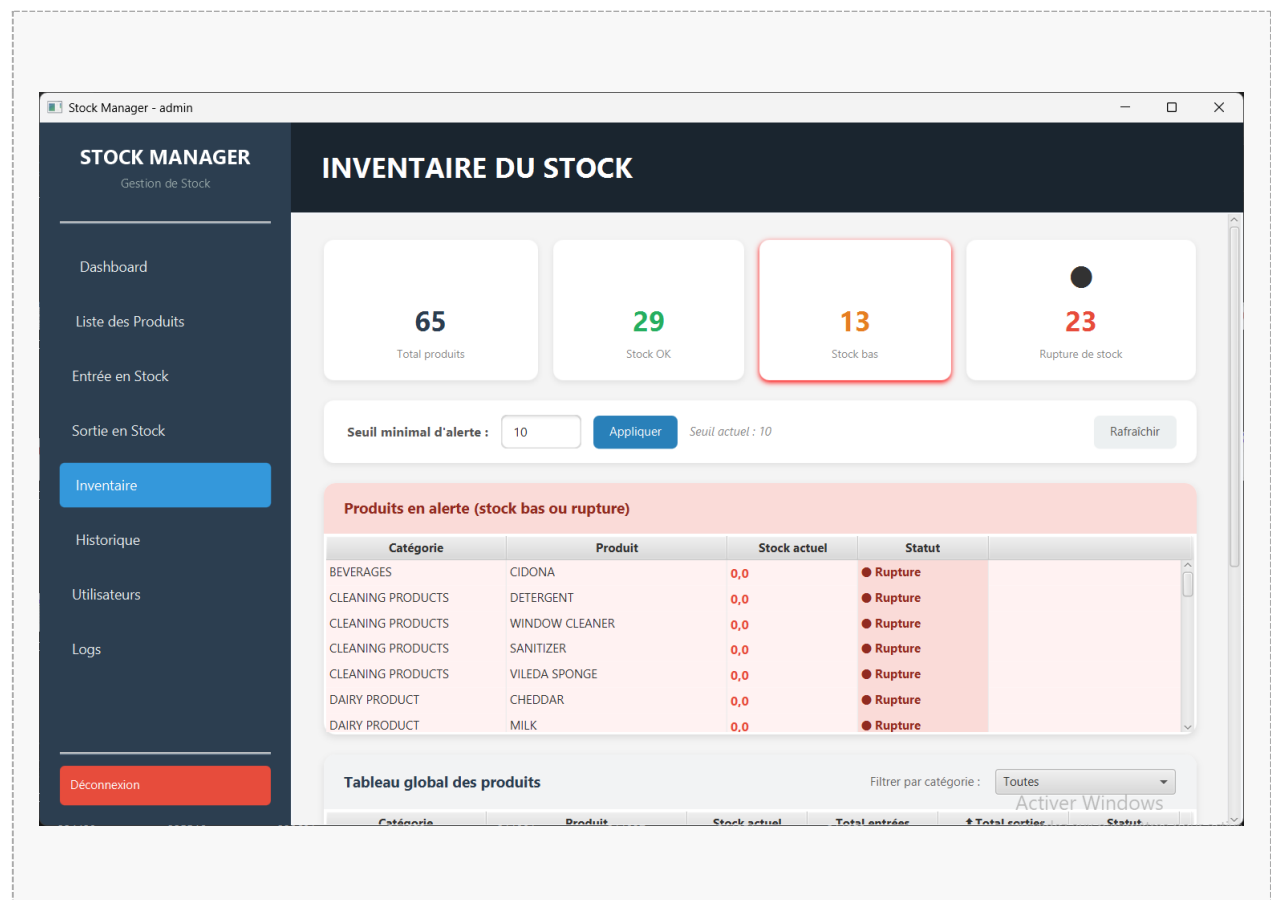


Figure 8 : Page Inventaire

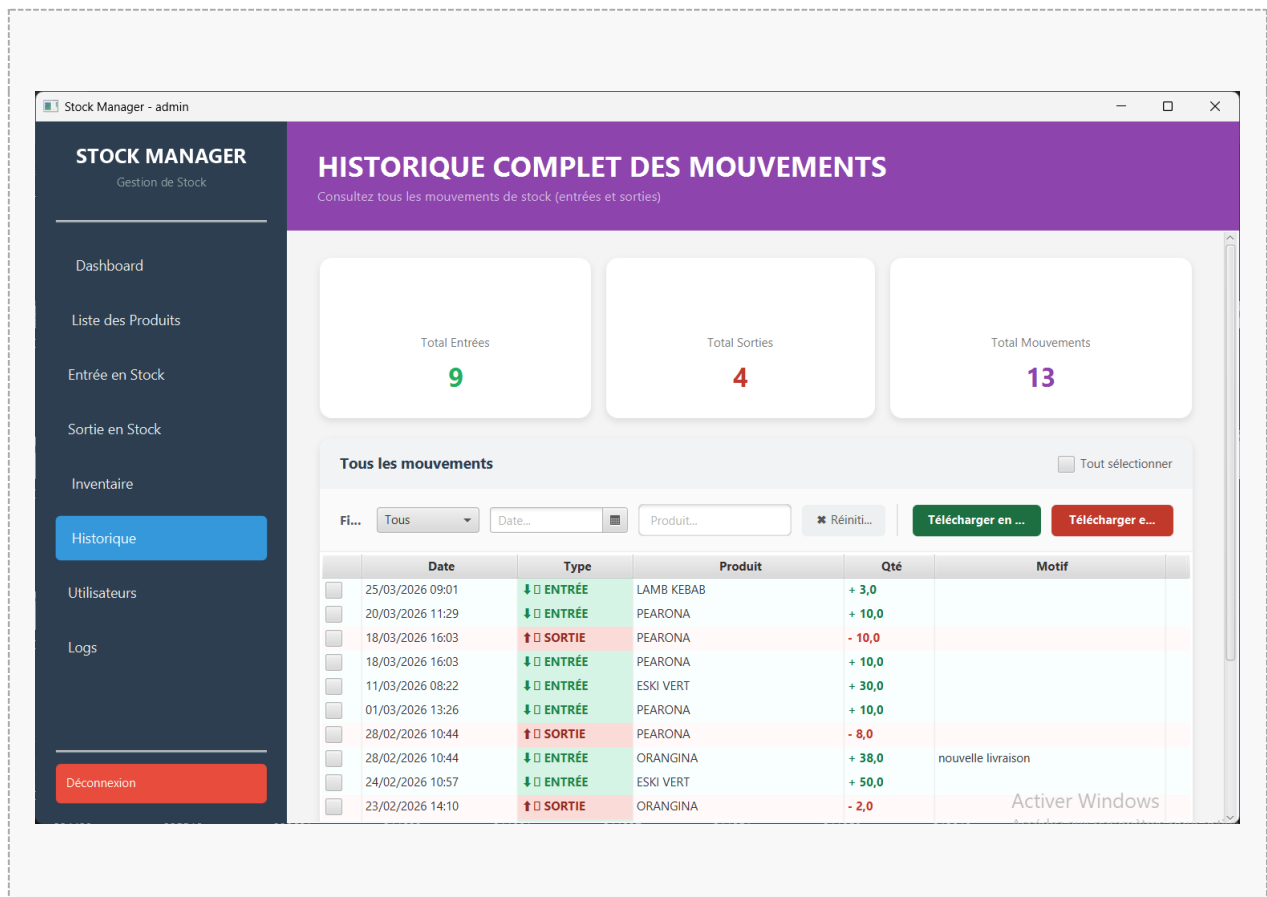


Figure 9 : Page Historique

L'application fonctionne en local sur chaque poste, mais la base de données étant centralisée en ligne, toutes les modifications sont immédiatement visibles pour les autres utilisateurs connectés. Cela résout le problème principal de l'ancienne organisation : plusieurs personnes pouvaient modifier le stock sans que les autres ne soient au courant.

B. Évaluation des résultats

Le bilan du projet est positif. L'ensemble des objectifs fixés dans le cahier des charges a été atteint, l'application a été livrée dans les délais et l'équipe du restaurant a commencé à l'utiliser avant la fin du stage. Les retours des premiers utilisateurs sont encourageants : ils apprécient notamment la rapidité de saisie des mouvements et la visibilité immédiate sur les stocks bas.

Quelques difficultés ont été rencontrées en cours de route. La synchronisation entre la base locale et la base en ligne a demandé plusieurs ajustements pour bien gérer les cas de coupure réseau. La gestion des rôles a également fait l'objet de discussions, car il a fallu trouver le bon équilibre entre sécurité et simplicité d'utilisation. Enfin, le travail en binôme a imposé une discipline de commits et de communication qui n'était pas toujours naturelle au départ, mais qui s'est révélée très formatrice.

Du point de vue de l'entreprise, le gain est concret : moins d'erreurs de saisie, meilleure visibilité sur les stocks, traçabilité complète des opérations et réduction du temps passé sur les inventaires mensuels.

CONCLUSION et PERSPECTIVES

Bilan personnel :

Ce stage a été une vraie expérience de projet de bout en bout. Il m'a permis de passer de la phase d'analyse des besoins à la livraison d'un outil fonctionnel, en passant par la modélisation, le développement, les tests et le déploiement. J'ai pu consolider mes compétences en Java et découvrir concrètement JavaFX, PostgreSQL et le pattern DAO, des outils que je ne connaissais qu'en théorie.

Sur le plan humain, le travail en binôme m'a appris à coordonner mon code avec celui d'un autre développeur, à gérer les conflits sur Git et à accepter que mes choix ne soient pas toujours ceux retenus. Les échanges avec la tutrice et avec l'équipe du restaurant m'ont également montré l'importance de bien écouter les utilisateurs finaux, qui ont souvent une vision du besoin différente de celle qu'on imagine au départ.

Perspectives :

L'application est aujourd'hui fonctionnelle, mais plusieurs pistes d'amélioration ont été identifiées pour les prochaines versions. Une première évolution serait l'ajout d'un module de gestion des fournisseurs, permettant de lier chaque entrée de stock à un fournisseur et d'en tirer des statistiques. Une seconde serait l'intégration d'un système de recettes, pour déduire automatiquement les ingrédients utilisés à chaque plat vendu.

Un tableau de bord plus avancé, avec des graphiques de consommation par période et par catégorie, serait également utile pour aider la direction à anticiper les commandes. Enfin, à plus long terme, une version mobile ou web pourrait être envisagée pour permettre à la direction de consulter les stocks depuis l'extérieur. Ces perspectives dépassent le cadre du stage, mais montrent que le projet a posé des bases solides sur lesquelles il est possible de continuer à construire.